

Distributed SpMV on Multi-GPU CUDA-Aware Systems

1st Guglielmo Boi

Department of Information Engineering and Computer Science

Università di Trento

Trento, Italy

guglielmo.boi@studenti.unitn.it

Abstract—This work investigates three distributed multi-GPU SpMV implementations based on 1D block partitioning, 1D cyclic partitioning, and 1D block partitioning with CUDA-aware MPI. The implementations were evaluated on SuiteSparse matrices using up to four NVIDIA A30 GPUs. The CUDA-aware MPI implementation provides no consistent benefit over conventional communication. Although cyclic partitioning outperforms the other approaches, all implementations exhibit limited scalability.

Index Terms—SpMV, CUDA, multi-GPU computing, 1D Partitioning, CUDA-Aware MPI, parallel computing.

I. INTRODUCTION

Sparse matrix-vector multiplication (SpMV) is a key linear algebra algorithm of great importance in computational science [1]. The application of SpMV is extensive and includes engineering computing, graph algorithms, and linear programming, among others [2].

Significant research progress has been made in accelerating SpMV on a single GPU. However, due to the limited memory capacity and computing capability, large scale SpMV computation still suffers from high latency on a single GPU [3].

As real world computing problems continue to increase in complexity, the need for parallel execution on multi-GPU systems is rapidly increasing.

Developing SpMV on multi-GPU systems introduces new challenges, such as effectively partitioning data for computational processes and reducing inter-process communication, the latter of which introduces significant transmission overhead.

The simplest 1D partitioning strategy evenly distributes the n rows among p processes in a block fashion, such that each process owns n/p consecutive rows, as shown in Figure 1.

An alternative 1D partition is the one shown in Figure 2, where the rows are assigned to the processes in a round-robin fashion.

This work compares the performance of three partitioning strategies for distributed SpMV: 1D cyclic partitioning, 1D block partitioning, and 1D block partitioning with CUDA-aware MPI memory transfers. A CPU implementation is also provided to test the correctness of the code.

II. METHODOLOGY

a) 1D Block Partitioning: In the 1D block partitioning strategy, blocks of consecutive rows are assigned to each

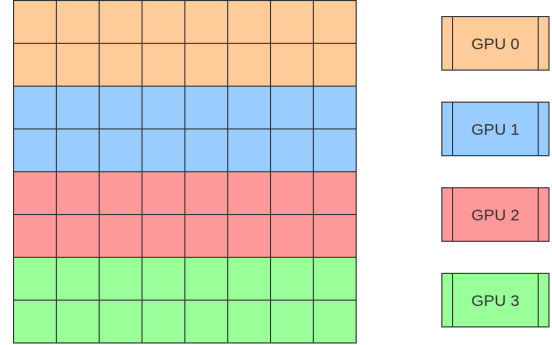


Fig. 1. Diagram showing the 1D block partitioning of a matrix.

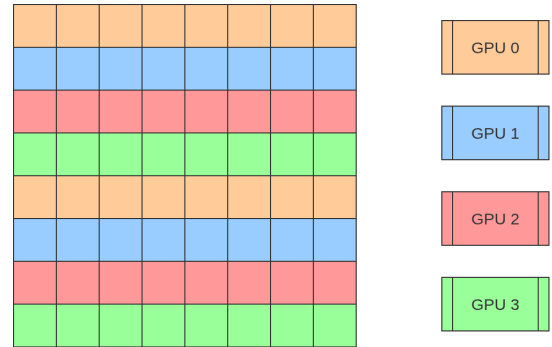


Fig. 2. Diagram showing the 1D cyclic partitioning of a matrix.

process. Each process stores its block of rows as a local CSR structure.

The contiguous row assignment favors the locality principle, reducing the volume of communication for matrices with banded structured sparsity patterns.

MPI_Scatter is used to distribute the matrix, while the results are gathered with MPI_Gather.

b) 1D Cyclic Partitioning: In the 1D cyclic partitioning strategy, process p is assigned rows $p, p + P, p + 2P, \dots$, where P is the total number of processes. This ensures an even distribution of rows regardless of their individual number of non-zero (NNZ) elements, which tends to balance the computational load when row lengths vary irregularly across the matrix.

With this partition a MPI Scatter approach is not possible,

Algorithm 1 1D Block Partitioning SpMV

```

1: Assign contiguous row block  $[r_{start}, r_{end})$  to each process
2: for each local row  $r$  from  $r_{start}$  to  $r_{end} - 1$  do
3:    $sum \leftarrow 0$ 
4:   for  $j = rowPtr[r]$  to  $rowPtr[r + 1] - 1$  do
5:      $sum \leftarrow sum + value[j] \cdot x[colIdx[j]]$ 
6:   end for
7:    $y_{local}[r] \leftarrow sum$ 
8: end for
9: Gather partitions of  $y$  via MPI_Gather

```

but point to point iterative communication must be implemented.

Algorithm 2 1D Cyclic Partitioning SpMV

```

1: Distribute rows to processes in round-robin order
2: for each local row  $r$  assigned to this process do
3:    $sum \leftarrow 0$ 
4:   for  $j = rowPtr[r]$  to  $rowPtr[r + 1] - 1$  do
5:      $sum \leftarrow sum + value[j] \cdot x[colIdx[j]]$ 
6:   end for
7:    $y_{local}[r] \leftarrow sum$ 
8: end for
9: Gather partitions of  $y$  via MPI_Gather

```

c) 1D Block Partitioning with CUDA-Aware MPI: This implementation replaces host-staged memory transfers with CUDA-aware MPI communication. In the original version, input data must be explicitly copied from device memory to a host buffer before MPI calls, and the received data must then be copied back to the device before the kernel launch.

With CUDA-aware MPI, device pointers can be passed directly to MPI calls, allowing the MPI runtime to manage the data movement internally. In principle, this should reduce transmission time overhead.

III. DATASET

a) SuiteSparse matrices: The evaluation uses two sets of matrices selected from the SuiteSparse Matrix Collection. The first set (strong dataset) consists of five matrices (*bcircuit*, *bcsstk14*, *boyd2*, *memplus*, and *scircuit*) and is used to evaluate the strong scalability of the implementation. The second set (weak dataset) consists of the *net* matrices, which are suitable for evaluating weak scalability.

The strong scalability dataset contains matrices with diverse sparsity structures and row distributions, whereas the *net* dataset provides progressively larger matrices with similar structural characteristics, allowing the scalability to be evaluated independently of changes in the sparsity pattern.

b) Random vectors: The input vectors of the SpMV operation are randomly generated by sampling floating-point values from a uniform distribution in the range $[0, 1000]$.

IV. EXPERIMENTAL SETUP

The multi-GPU implementations were tested with up to 4 NVIDIA A30 GPUs with 24 GB of memory each. The analysis

TABLE I
SELECTED SUITESPARSE MATRICES USED IN THE EVALUATION

Matrix	Size	NNZ	Symmetry	Sparsity
bcircuit	68,902	375,558	Unsymmetric	Slight
bcsstk14	1,806	63,454	Symmetric	Moderate
boyd2	466,385	1,502,827	Unsymmetric	Extreme
memplus	17,758	99,147	Unsymmetric	Moderate
scircuit	170,998	958,936	Unsymmetric	Slight
net25	9,520	401,200	Symmetric	Moderate
net50	16,320	945,200	Symmetric	Moderate
net75	23,120	1,489,200	Symmetric	Moderate
net100	29,920	2,033,200	Symmetric	Moderate
net125	36,720	2,577,200	Symmetric	Moderate
net150	43,520	3,121,200	Symmetric	Moderate

focuses on execution time, FLOPS, speedup, and efficiency. The SpMV number of floating operations is conventionally measured as $2 \cdot NNZ$. Several runs were performed: mean and standard deviation (SD) values are reported.

V. RESULTS

a) Execution Time: Table II reports the mean execution time and standard deviation for all multi-GPU implementations across both the 2 and 4 GPUs configurations.

The 1D cyclic partitioning consistently achieves the best results. At 4 GPUs, cyclic outperforms block partitioning for all matrices except *bcsstk14*, with improvements ranging from approximately 5.5% on *scircuit* to nearly 60% on *bcircuit*.

On the *net* dataset the advantage is stable and substantial, with cyclic reducing execution time by up to 37% compared to 1D block across all six matrices. This outcome is explained by the round-robin row assignment, which distributes non-zero elements more evenly across processes, balancing the computational load and reducing idle time.

The 1D block partitioning with CUDA-aware MPI does not deliver consistent improvements over the standard 1D block implementation. At 4 GPUs, CUDA-aware MPI achieves a modest reduction for *bcircuit*, but increases execution time for most other matrices, with particularly large penalties on *bcsstk14* and *memplus*. The high standard deviation values observed for CUDA-aware MPI at 4 GPUs further indicate instability in the inter-GPU data transfers.

Scaling from 2 to 4 GPUs shows clear improvement for larger matrices under block and cyclic partitioning: *boyd2*, for instance, drops from 78.5 ms to 50.6 ms with 1D block and from 57.6 ms to 39.2 ms with 1D cyclic. However, for smaller matrices such as *bcircuit* and *bcsstk14*, going from 2 to 4 GPUs can increase execution time under block partitioning, suggesting that the synchronization and communication costs outweigh parallelism gains when the input size is small.

b) FLOPS: Figure 3 reports the MFLOPS achieved by each implementation on 4 GPUs.

The cyclic partitioning yields the highest throughput across all matrices, with averages of 38.7 MFLOPS on the strong

TABLE II
TOTAL EXECUTION TIME (MS) COMPARISON ACROSS MULTI-GPU IMPLEMENTATIONS USING 2 AND 4 GPUS.

Matrix	1D Block				1D Cyclic				1D Block CUDA-Aware			
	2 GPUs		4 GPUs		2 GPUs		4 GPUs		2 GPUs		4 GPUs	
	Mean	SD	Mean	SD	Mean	SD	Mean	SD	Mean	SD	Mean	SD
bcircuit	40.964	3.356	46.870	2.521	24.725	0.723	18.808	1.166	28.468	1.506	39.310	17.491
bcsstk14	5.198	1.933	7.663	10.004	4.644	1.055	8.358	15.192	6.377	0.844	27.102	18.106
boyd2	78.505	1.096	50.595	1.442	57.576	1.223	39.207	1.527	74.829	3.379	52.273	1.622
memplus	11.586	1.553	8.718	0.931	8.160	0.347	6.234	0.120	10.202	0.517	16.946	4.694
scircuit	56.932	0.964	40.867	1.228	55.033	0.565	38.628	0.563	55.879	0.429	47.543	10.496
net25	21.093	0.592	13.664	0.354	12.238	0.318	10.227	1.866	20.861	0.643	18.131	3.104
net50	38.285	0.513	28.131	0.860	26.509	0.096	17.779	1.486	39.212	1.592	33.464	6.567
net75	54.427	1.025	44.939	0.616	42.717	1.025	29.357	1.184	54.426	2.432	47.918	3.306
net100	72.518	0.941	63.272	0.815	60.212	0.254	40.443	1.522	75.614	1.196	66.309	3.963
net125	92.073	0.892	80.369	0.956	79.025	1.011	50.571	0.811	91.079	1.719	81.361	2.693
net150	101.932	0.653	92.163	0.826	92.645	0.261	61.024	1.362	104.311	1.832	94.681	2.871

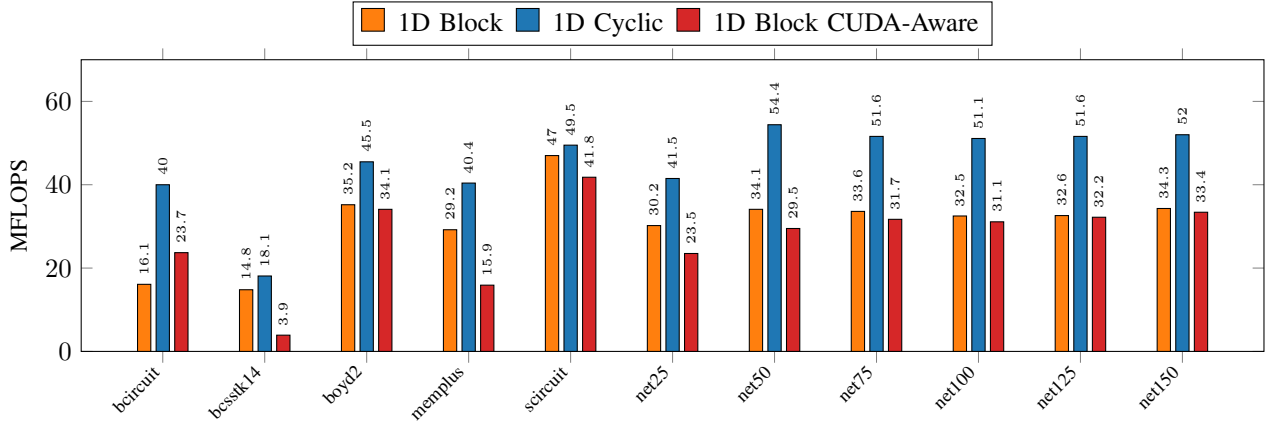


Fig. 3. Total MFLOPS comparison across multi-GPU implementations using 4 GPUs. FLOPS are measured as $\frac{2 \cdot \text{NNZ}}{\text{Total Time}}$.

dataset and 50.4 MFLOPS on the weak dataset. Block partitioning achieves 28.5 MFLOPS and 32.9 MFLOPS respectively, while CUDA-aware MPI reaches only 23.9 MFLOPS and 30.2 MFLOPS.

On the net matrices, the cyclic implementation sustains over 50 MFLOPS from `net50` through `net150`, reflecting the benefit of its load-balanced partitioning as matrix size grows. In contrast, block and CUDA-aware MPI plateau below 35 MFLOPS for the same matrices.

Among the matrices of the strong dataset, `scircuit` achieves the highest throughput overall, reaching 49.5 MFLOPS under cyclic and 47.0 MFLOPS under block. The CUDA-aware MPI implementation consistently underperforms relative to standard block partitioning, confirming that the latency introduced by device transfers outweighs the time saving from eliminating explicit host staging at this modest scale.

VI. DISCUSSION

a) Speedup: Figure 4 shows the average speedup over the two datasets. The speedup is calculated by using the 1 GPU configuration values as baseline for each implementation. All implementations achieve underwhelming results and the gap with the ideal curve is significant across all configurations.

On the strong dataset, 1D cyclic is the only implementation that exceeds a speedup of 1. 1D block remains close to 1 at 4 GPUs, meaning that on average it barely matches the performance of a single GPU. CUDA-aware MPI degrades consistently, falling to $0.77\times$ at 2 GPUs and $0.66\times$ at 4 GPUs, indicating that the overhead from device transfers outweighs any benefit from eliminating explicit host staging. None of the implementations shows strong scalability.

The weak scalability results follow a similar pattern. 1D cyclic reaches the highest average speedup of $1.38\times$ at 4 GPUs. The fact that cyclic exceeds a speedup of 1 on the weak dataset despite increasing problem size suggests that the load balancing benefit grows with matrix size: as the net matrices scale up, the round-robin distribution maintains an even NNZ distribution across GPUs, keeping all processes equally busy and reducing synchronization stalls.

b) Efficiency: Figure 5 reports efficiency, defined as speedup divided by number of GPUs. All implementations show a drop in efficiency as GPU count increases.

On the strong dataset at 2 GPUs, cyclic achieves the best efficiency at 0.51, followed by block at 0.43 and CUDA-aware MPI at 0.39. The drop from 2 to 4 GPUs is caused by the fixed problem size, meaning that each GPU receives a smaller local workload, and the cost of communication becomes an

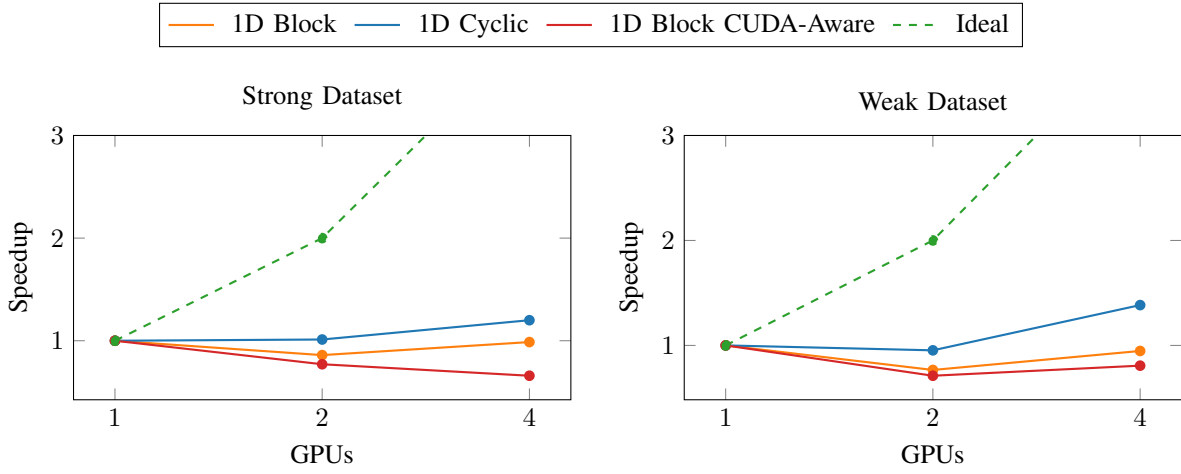


Fig. 4. Speedup over number of GPUs, across the different multi-GPU implementations.

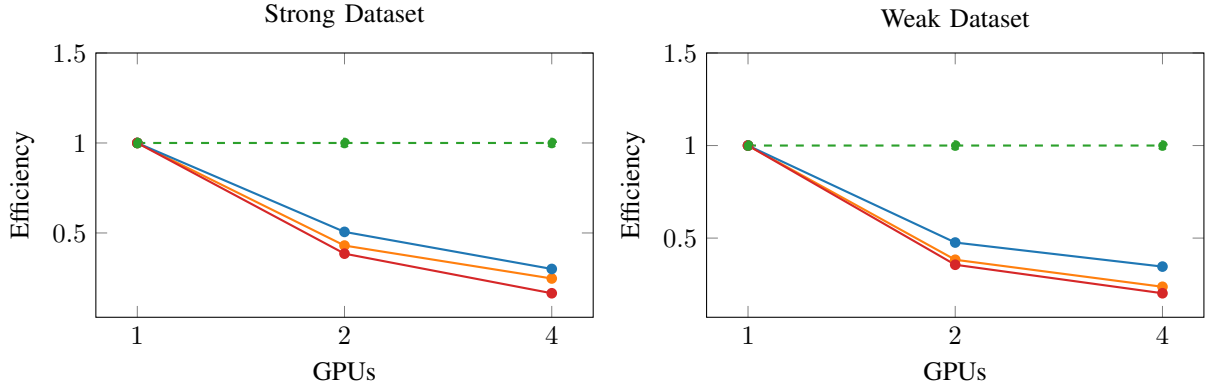


Fig. 5. Efficiency over number of GPUs, across the different multi-GPU implementations.

increasingly large proportion of the total time.

The weak dataset shows comparable or slightly lower efficiency values. This suggests that the additional communication volume introduced by the larger net matrices matches the benefit of proportionally increasing the problem size.

Overall, the low efficiency values observed across all implementations highlight that the communication to computation ratio is unfavorable at this scale. The reason is that SpMV is a memory-bound problem, meaning that data transfers between GPUs, whether host-staged or CUDA-aware, represent a significant cost relative to the computation performed, as shown by the results.

VII. CONCLUSIONS

This work presented and evaluated three distributed multi-GPU implementations of SpMV: 1D block partitioning, 1D cyclic partitioning, and 1D block partitioning with CUDA-aware MPI. The results show that the 1D cyclic partitioning consistently provides the best performance. Its round-robin row distribution improves load balancing, particularly for matrices with irregular non-zero distributions.

The CUDA-aware MPI implementation did not provide the expected performance benefits on the evaluated platform,

because the communication overhead often resulted in slower execution than the standard implementation.

Overall, the experiments confirm that communication costs are the primary limitation to multi-GPU SpMV scalability. Future work could investigate 2D matrix partitioning strategies and optimized communication patterns to improve scalability.

REFERENCES

- [1] Chu, G., He, Y., Dong, L., Ding, Z., Chen, D., Bai, H., ... & Hu, C. (2023, August). Efficient algorithm design of optimizing spmv on gpu. In Proceedings of the 32nd international symposium on high-performance parallel and distributed computing (pp. 115-128).
- [2] Gao, J., Liu, B., Ji, W., & Huang, H. (2024). A systematic literature survey of sparse matrix-vector multiplication. arXiv preprint arXiv:2404.06047.
- [3] Gao, J., Ji, W., & Wang, Y. (2024). Optimization of large-scale sparse matrix-vector multiplication on multi-GPU systems. ACM Transactions on Architecture and Code Optimization, 21(4), 1-24.
- [4] Boman, E. G., Devine, K. D., & Rajamanickam, S. (2013, November). Scalable matrix computations on large scale-free graphs using 2D graph partitioning. In Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (pp. 1-12).